PROGRAMACIÓN DE SERVICIOS Y PROCESOS - 2º DAM

I.E.S. ALONSO DE AVELLANEDA (ALCALÁ DE HENARES)
EVALUACIÓN: 1ª Evaluación | **FECHA:** Convocatoria Ordinaria
NOMBRE Y APELLIDOS: _____
CALIFICACIÓN: _____ / 10

Esta prueba evalúa los siguientes resultados de aprendizaje:

- **RA1:** Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela.
- **RA2:** Desarrolla aplicaciones compuestas por varios hilos de ejecución analizando y aplicando librerías específicas del lenguaje de programación (`Thread`, `yield`, `synchronized`, `wait`, `notifyAll`, patrón productor-consumidor).

Duración:

- **Tiempo estándar:** 115 minutos (1 hora y 55 minutos).
- **Adaptación NEAE (+10%):** 126 minutos (2 horas y 6 minutos).

TAREAS A REALIZAR

Una empresa de distribución y logística automatizada precisa desarrollar un prototipo para la recepción, clasificación y embalaje de paquetes en su centro nodriza de distribución.

El ciclo de trabajo del sistema se estructura en tres fases:

1. **Recepción y descarga de mercancía (Procesos):** Llegada y descarga de furgonetas de reparto implementadas como procesos hijo independientes del sistema operativo.
2. **Cinta clasificadora con Patrón Productor-Consumidor (Hilos y Monitor):** Transferencia de los paquetes desde los puntos de descarga a una cinta transportadora de capacidad finita protegida por un monitor con `wait()` y `notifyAll()`.
3. **Embalaje cooperativo con cesión de CPU (`yield`):** Varios hilos operarios de embalaje retiran paquetes de la cinta y aplican `Thread.yield()` tras cada operación para favorecer una alternancia equilibrada en la planificación del procesador.

La solución se divide en **dos aplicaciones independientes**:

---

1. Aplicación Centro Logístico (Coordinador y Clasificador)

- **Parámetros de entrada:** Recibe el número de furgonetas de reparto a simular (procesos hijo) por argumento de línea de comandos (`args[0]`) o mediante lectura de teclado (`Scanner`). Si el dato no es un entero positivo válido, se asumirá el valor de **4 furgonetas por defecto**.
- **Gestión de procesos hijo (`ProcessBuilder`):**
  - Lanza de forma concurrente o secuencial los procesos furgoneta.
  - Debe ser plenamente compatible con Linux y Windows (localizando `java` vía `which java` / `where java` o directamente mediante el ejecutable en el `PATH`).
  - Lee por la tubería de entrada estándar el identificador de cada furgoneta emitido por el proceso hijo y almacena su referencia.
  - La aplicación principal espera a que finalicen todas las furgonetas utilizando el método **`waitFor()`** antes de activar la cinta clasificadora.
- **Monitor `CintaTransportadora` (Patrón Productor-Consumidor):**
  - Representa un búfer compartido con una **capacidad limitada a un máximo de 3 paquetes simultáneos**.
  - Métodos sincronizados obligatorios:
    - `public synchronized void depositar(String paquete)`: Si la cinta está llena (3 paquetes), el hilo productor se pone en espera invocando **`wait()` en un bucle `while`**. Tras depositar el paquete en el búfer, invoca obligatoriamente **`notifyAll()`** para despertar a los consumidores.
    - `public synchronized String retirar()`: Si la cinta está vacía (0 paquetes), el hilo consumidor se pone en espera invocando **`wait()` en un bucle `while`**. Tras retirar un paquete, invoca obligatoriamente **`notifyAll()`** para despertar a los productores.
- **Hilos Productores (Descargadores):**
  - Se lanzan hilos encargados de leer los ficheros de paquetes dejados por las furgonetas y depositarlos en el monitor `CintaTransportadora`.
- **Hilos Consumidores (Operarios de Embalaje):**
  - Retiran paquetes de la cinta y los contabilizan según su código de zona en un registro estadístico protegido con exclusión mutua.
  - **Uso obligatorio de `Thread.yield()`:** Tras procesar con éxito un paquete, cada hilo consumidor invocará explícitamente `Thread.yield()` (o su equivalente `Thread.sleep(0)`) para sugerir al planificador de la CPU que ceda su turno a otros hilos de igual prioridad, evitando que un único hilo acapare la cinta.

- **Control de incidencias:** Si alguna furgoneta no generó fichero de paquetes (incidencia en ruta), se registrará debidamente sin provocar excepciones no controladas.
- **Cierre y sincronización final:** La aplicación principal asegura la finalización de los hilos de procesamiento mediante **`join()`** y presenta el informe final consolidado.

---

2. Aplicación Furgoneta de Reparto (Proceso Hijo)

- **Identificación:**
  - Al iniciar su ejecución, espera un tiempo aleatorio entre **0 y 400 milisegundos**.
  - Construye su código identificador con una marca de tiempo del sistema: `System.currentTimeMillis()`.
  - Informa al proceso padre enviando su identificador a través de la salida estándar (`System.out.println(id_furgoneta)`).
- **Descarga de paquetes:**
  - Simula las labores de muelle y descarga durmiendo el hilo entre **1 y 2,5 segundos** (`Thread.sleep`).
- **Probabilidad de avería/retraso (Sin entrega):**
  - De media, **1 de cada 5 furgonetas (20% de probabilidad)** sufrirá una avería en ruta, por lo que finalizará sin generar ningún fichero de paquetes.
- **Generación del fichero de manifiesto:**
  - Las furgonetas operativas generan un número aleatorio de paquetes (entre 2 y 4 paquetes).
  - Cada paquete tiene asignado un código numérico de destino entre **1 y 5** (ver tabla).
  - Crea un fichero denominado `<id_furgoneta>.txt` con un paquete por línea indicando su código numérico.

Tabla de Códigos de Destino de Paquetería

Código Numérico	Tipo de Envío	Zona de Distribución
1	Local	Madrid Urbana
2	Provincial	Comunidad de Madrid
3	Peninsular	Nacional España
4	Islas	Baleares y Canarias
5	Internacional	Unión Europea

EJEMPLO DE EJECUCIÓN

Consola de la Aplicación Centro Logístico (Ejecución con 5 furgonetas)

```
None
Indica el número de furgonetas de reparto: 5
** Desplegando 5 procesos de furgonetas... **
 -> Muelle asignado a furgoneta: FURGO-1731095901120
 -> Muelle asignado a furgoneta: FURGO-1731095901254
 -> Muelle asignado a furgoneta: FURGO-1731095901398
 -> Muelle asignado a furgoneta: FURGO-1731095901511
 -> Muelle asignado a furgoneta: FURGO-1731095901680

** Todas las furgonetas han concluido su descarga. **
** Activando cinta transportadora y operarios de embalaje... **

[PRODUCTOR] Paquete 'Paq-Nacional-3' depositado en cinta. Ocupación: 1/3
[PRODUCTOR] Paquete 'Paq-Local-1' depositado en cinta. Ocupación: 2/3
[CONSUMIDOR 1] Paquete 'Paq-Nacional-3' retirado de cinta. Ocupación: 1/3
[CONSUMIDOR 1] Embalaje completado. Invocando yield() para alternar CPU...
[PRODUCTOR] Paquete 'Paq-Internacional-5' depositado en cinta. Ocupación: 2/3
[CONSUMIDOR 2] Paquete 'Paq-Local-1' retirado de cinta. Ocupación: 1/3
[CONSUMIDOR 2] Embalaje completado. Invocando yield() para alternar CPU...
[PRODUCTOR] Paquete 'Paq-Provincial-2' depositado en cinta. Ocupación: 2/3
[PRODUCTOR] Paquete 'Paq-Islas-4' depositado en cinta. Ocupación: 3/3
[PRODUCTOR] Cinta LLENA (3/3). Productor entra en wait()...
[CONSUMIDOR 1] Paquete 'Paq-Internacional-5' retirado de cinta. Ocupación: 2/3 ->
notifyAll() emitido.
[CONSUMIDOR 1] Embalaje completado. Invocando yield() para alternar CPU...

============================================================
         BALANCE DE DISTRIBUCIÓN Y EMBALAJE
============================================================
Furgonetas esperadas   : 5
Furgonetas procesadas  : 4
Furgonetas con avería  : 1

Desglose de paquetes embalados:
  - Destino 1 (Local - Madrid Urbana)       : 3 paquetes
  - Destino 2 (Provincial - Comunidad Madrid): 2 paquetes
  - Destino 3 (Peninsular - Nacional)       : 4 paquetes
  - Destino 4 (Islas - Baleares y Canarias) : 1 paquetes
  - Destino 5 (Internacional - UE)          : 2 paquetes
```

```
Total de paquetes procesados con éxito: 12
============================================================
```

CONSIDERACIONES A LA HORA DE VALORAR EL EJERCICIO

- **Compilación estricta:** La solución debe compilar obligatoriamente sin errores de sintaxis. **Un código que no compile no podrá obtener más de 2 puntos sobre 10.**
- **Entorno de examen:** Prohibido el uso de redes externas, navegadores e Inteligencia Artificial durante la prueba.
- **Buenas prácticas:** Modularidad adecuada en clases, métodos autocontenidos, gestión estricta de variables compartidas y comentarios claros de concurrencia.
- **Sincronización:** El uso de `wait()` debe situarse obligatoriamente dentro de bucles `while`, y la reactivación de hilos debe realizarse mediante `notifyAll()`.
- **Portabilidad multiplataforma:** El código debe ser ejecutable tanto en Linux como en Windows sin alterar rutas ni romper la ejecución por separadores de archivos o fin de línea.

RÚBRICA DE CALIFICACIÓN (TOTAL: 10 PUNTOS)

1. Aplicación Centro Logístico (Total: 6 puntos)

- **1.0 pto:** Validación de entrada con valor por defecto de 4 y lectura robusta de argumentos o teclado.
- **1.0 pto:** Lanzamiento de procesos hijo con `ProcessBuilder`, captura del flujo de salida (`InputStream`) y espera con `waitFor()`.
- **2.0 ptos:** Implementación correcta del monitor `CintaTransportadora` con exclusión mutua (`synchronized`), capacidad máxima de 3 elementos, invocación de `wait()` en bucles `while` y reactivación mediante `notifyAll()`.
- **1.0 pto:** Creación y coordinación de hilos productores y consumidores, incluyendo el uso explícito de `Thread.yield()` en los consumidores tras procesar cada paquete.
- **1.0 pto:** Sincronización final con `join()`, recuento protegido de paquetes por zonas y presentación del informe formateado.

2. Aplicación Furgoneta de Reparto / Proceso Hijo (Total: 2 puntos)

- **1.0 pto:** Construcción del identificador con timestamp, notificación por consola estándar y retardo simulado con `Thread.sleep`.

- **1.0 pto:** Simulación probabilística de avería en ruta (20%) y generación correcta del fichero de texto `<id_furgoneta>.txt` con los paquetes.

3. Concurrencia y Calidad del Diseño (Total: 2 puntos)

- **1.0 pto:** Flujo limpio y sin interbloqueos (*deadlocks*) entre productores y consumidores sobre la cinta transportadora.
- **1.0 pto:** Calidad global del código, modularidad, tratamiento de excepciones comprobadas (`IOException`, `InterruptedException`) y compatibilidad Linux / Windows.